Search the docs ...

Array objects

Array API Standard Compatibility

Constants

Universal functions (`ufunc`)

Routines

Typing (`numpy.typing`)

Global state

Packaging (`numpy.distutils`)

NumPy distutils - users guide

Status of `numpy.distutils` and migration advice

NumPy C-API

CPU/SIMD Optimizations

NumPy security

NumPy and SWIG

Constants

NumPy includes several constants:

numpy.Inf

IEEE 754 floating point representation of (positive) infinity.

Use `inf` because `Inf`, `Infinity`, `PINF` and `infty` are aliases for `inf`. For more details, see `inf`.

See Also

inf

numpy.Infinity

IEEE 754 floating point representation of (positive) infinity.

Use `inf` because `Inf`, `Infinity`, `PINF` and `infty` are aliases for `inf`. For more details, see `inf`.

See Also

inf

numpy.NAN

IEEE 754 floating point representation of Not a Number (NaN).

`NaN` and `NAN` are equivalent definitions of `nan`. Please use `nan` instead of `NAN`.

See Also

nan

numpy.NINF

IEEE 754 floating point representation of negative infinity.

Returns

y : *float*
A floating point representation of negative infinity.

See Also

isinf : Shows which elements are positive or negative infinity

isposinf : Shows which elements are positive infinity

isneginf : Shows which elements are negative infinity

isnan : Shows which elements are Not a Number

isfinite : Shows which elements are finite (not one of Not a Number, positive infinity and negative infinity)

Notes

NumPy uses the IEEE Standard for Binary Floating-Point for Arithmetic (IEEE 754). This means that Not a Number is not equivalent to infinity. Also that positive infinity is not equivalent to negative infinity. But infinity is equivalent to positive infinity.

Examples

```
>>> np.NINF
-inf
>>> np.log(0)
-inf
```

numpy.NZERO

IEEE 754 floating point representation of negative zero.

Returns

y : *float*
A floating point representation of negative zero.

See Also

PZERO : Defines positive zero.

isinf : Shows which elements are positive or negative infinity.

isposinf : Shows which elements are positive infinity.

isneginf : Shows which elements are negative infinity.

isnan : Shows which elements are Not a Number.

isfinite : *Shows which elements are finite - not one of*
Not a Number, positive infinity and negative infinity.

Notes

NumPy uses the IEEE Standard for Binary Floating-Point for Arithmetic (IEEE 754). Negative zero is considered to be a finite number.

Examples

```
>>> np.NZERO
-0.0
>>> np.PZERO
0.0
```

```
>>> np.isfinite([np.NZERO])
array([ True])
>>> np.isnan([np.NZERO])
array([False])
>>> np.isinf([np.NZERO])
array([False])
```

numpy.NaN

IEEE 754 floating point representation of Not a Number (NaN).

NaN and NAN are equivalent definitions of nan. Please use nan instead of NaN.

See Also

nan

numpy.PINF

IEEE 754 floating point representation of (positive) infinity.

Use inf because Inf, Infinity, PINF and infty are aliases for inf. For more details, see inf.

Search the docs ...

Array objects
The N-dimensional array (ndarray)
Scalars
numpy.generic.flags
numpy.generic.shape
numpy.generic.strides
numpy.generic.ndim
numpy.generic.data
numpy.generic.size
numpy.generic.itemsize
numpy.generic.base
numpy.generic.dtype
numpy.generic.real
numpy.generic.imag
numpy.generic.flat
numpy.generic.T
numpy.generic.__array_int
numpy.generic.__array_str
numpy.generic.__array_pri
numpy.generic.__array_wra
numpy.generic.__array__
numpy.generic.__array_wra
numpy.generic.squeeze
numpy.generic.byteswap
numpy.generic.__reduce__
numpy.generic.__setstate__
numpy.generic.setflags
numpy.number.__class_ge
Data type objects (dtype)
Indexing routines
Iterating over arrays
Standard array subclasses
Masked arrays
The array interface protocol
Datetimes and Timedeltas
Array API Standard Compatibility
Constants
Universal functions (ufunc)
Routines
Typing (numpy.typing)
Global state
Packaging (

Scalars

Python defines only one type of a particular data class (there is only one integer type, one floating-point type, etc.). This can be convenient in applications that don't need to be concerned with all the ways data can be represented in a computer. For scientific computing, however, more control is often needed.

In NumPy, there are 24 new fundamental Python types to describe different types of scalars. These type descriptors are mostly based on the types available in the C language that CPython is written in, with several additional types compatible with Python's types.

Array scalars have the same attributes and methods as **ndarrays**. [1] This allows one to treat items of an array partly on the same footing as arrays, smoothing out rough edges that result when mixing scalar and array operations.

Array scalars live in a hierarchy (see the Figure below) of data types. They can be detected using the hierarchy: For example, `isinstance(val, np.generic)` will return **True** if *val* is an array scalar object. Alternatively, what kind of array scalar is present can be determined using other members of the data type hierarchy. Thus, for example `isinstance(val, np.complexfloating)` will return **True** if *val* is a complex valued type, while `isinstance(val, np.flexible)` will return true if *val* is one of the flexible itemsize array types (**str_**, **bytes_**, **void**).

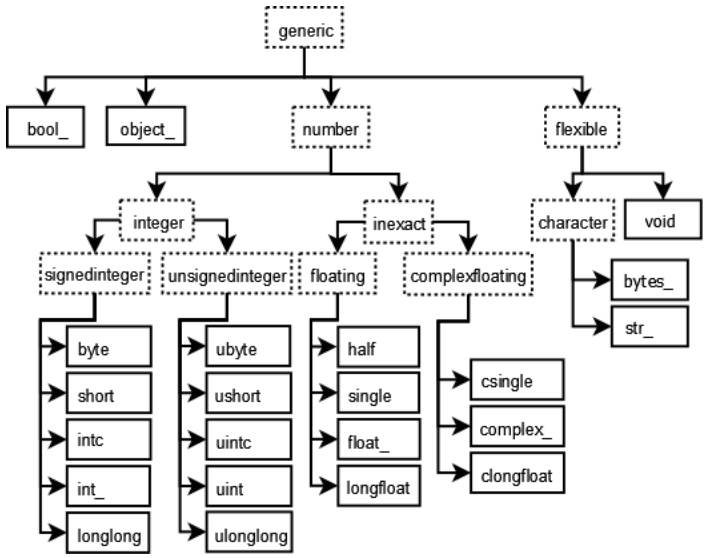


Figure: Hierarchy of type objects representing the array data types. Not shown are the two integer types **intp** and **uintp** which just point to the integer type that holds a pointer for the platform. All the number types can be obtained using bit-width names as well.

[1] However, array scalars are immutable, so none of the array scalar attributes are settable.

Built-in scalar types

The built-in scalar types are shown below. The C-like names are associated with character codes, which are shown in their descriptions. Use of the character codes, however, is discouraged.

Some of the scalar types are essentially equivalent to fundamental Python types and therefore inherit from them as well as from the generic array scalar type:

class numpy.**single** [source]

Single-precision floating-point number type, compatible with C `float`.

Character code:	`'f'`
Alias on this platform (Linux x86_64):	`numpy.float32`: 32-bit-precision floating-point number type: sign bit, 8 bits exponent, 23 bits mantissa.

class numpy.**double**(*x=0, /*) [source]

Double-precision floating-point number type, compatible with Python *float* and C `double`.

Character code:	`'d'`
Alias:	`numpy.float_`
Alias on this platform (Linux x86_64):	`numpy.float64`: 64-bit precision floating-point number type: sign bit, 11 bits exponent, 52 bits mantissa.

class numpy.**longdouble** [source]

Extended-precision floating-point number type, compatible with C `long double` but not necessarily with IEEE 754 quadruple-precision.

Character code:	`'g'`
Alias:	`numpy.longfloat`
Alias on this platform (Linux x86_64):	`numpy.float128`: 128-bit extended-precision floating-point number type.

Complex floating-point types

class numpy.**complexfloating** [source]

Abstract base class of all complex number scalar types that are made up of floating-point numbers.

class numpy.**csingle** [source]

Complex number type composed of two single-precision floating-point numbers.

Character code:	`'F'`
Alias:	`numpy.singlecomplex`
Alias on this platform (Linux x86_64):	`numpy.complex64`: Complex number type composed of 2 32-bit-precision floating-point numbers.

class numpy.**cdouble**(*real=0, imag=0*) [source]

Complex number type composed of two double-precision floating-point numbers, compatible with Python *complex*.

Character code:	`'D'`
Alias:	`numpy.cfloat`
Alias:	`numpy.complex_`
Alias on this platform (Linux x86_64):	`numpy.complex128`: Complex number type composed of 2 64-bit-precision floating-point numbers.

class numpy.**clongdouble** [source]

Complex number type composed of two extended-precision floating-point numbers.

Character code:	`'G'`
Alias:	`numpy.clongfloat`
Alias:	`numpy.longcomplex`
Alias on this platform (Linux x86_64):	`numpy.complex256`: Complex number type composed of 2 128-bit extended-precision floating-point numbers.

Other types

class numpy.**bool_** [source]